

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CLASS TEST

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- CLASS TEST
Weightage:	30%	Total Marks:	25
C05 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	V ADARSH REDDY	Reg.No.:	192525412
Department:	B TECH AI/ML	Date:	26-08-2026

ANNEXURE A

CLASS TEST

1. Design a heterogeneous I/O bus architecture comparing USB, PCIe, and Thunderbolt, and construct a scalable communication model for multiple peripherals.
2. Construct an interrupt handling framework for a multi-core processor comparing vectored interrupts and shared interrupt lines, and analyze latency impact.
3. Design a high-speed NVMe storage subsystem using PCIe lanes, DMA transfers, and memorymapped registers, and justify performance improvements.
4. Construct an I/O scheduling mechanism comparing cycle stealing DMA and burst mode DMA for continuous multimedia streaming.

5. Design a multi-level I/O communication architecture combining memory-mapped I/O, interrupt handling, and advanced bus interfaces, and evaluate throughput.

Code of Conduct:

I V ADARSH REDDY (192525412) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V ADARSH REDDY

Signature of the student:

MARKS ALLOCATION

	Conceptual Understanding (30%)	Accuracy of Answers (30%)	Problem Solving & Application Skills (20%)	Presentation & Organization (10%)	Completion & Time Management (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30%	Demonstrates excellent understanding of all concepts with accurate explanations and applications	Shows good understanding of concepts with minor errors	Shows basic understanding but with some misconceptions	Shows limited understanding with major conceptual errors

Accuracy of Answers	30%	Provides completely correct answers with proper steps, logic, and final solutions	Provides mostly correct answers with few minor mistakes	Provides partially correct answers with missing steps/errors	Provides incorrect or incomplete answers with minimal accuracy
Problem Solving & Application Skills	20%	Applies appropriate methods/techniques effectively and solves complex problems independently	Applies suitable methods with minor guidance/errors	Attempts problem solving but has difficulty applying concepts correctly	Unable to apply concepts or select suitable methods
Presentation & Organization	10%	Answers are well structured, neat, and clearly presented with proper notation/format	Answers are organized with minor presentation issues	Answers lack proper organization and clarity in some areas	Poorly organized answers with unclear presentation
Completion & Time Management	10%	Completes all questions within the allotted time with effective planning	Completes most questions within time	Attempts only part of the test due to time management issues	Very few questions attempted or incomplete submission

Q1. Design a heterogeneous I/O bus architecture comparing USB, PCIe, and Thunderbolt, and construct a scalable communication model for multiple peripherals.

Bus Comparison

Feature	USB (3.2/4)	PCIe (Gen4/5)	Thunderbolt (4/5)
Topology	Tiered-star, host-centric	Point-to-point, tree of switches	Daisy-chain over PCIe + DisplayPort tunnel
Max Bandwidth	Up to 20-40 Gbps	Up to 64 GT/s per lane (Gen5)	Up to 80-120 Gbps
Latency	Moderate, polling/interrupt based	Low, memory-mapped, DMA capable	Low, tunnels PCIe + DP
Addressing	Endpoint/pipe based	Memory/IO address space	PCIe address space (tunneled)

Typical Use	Peripherals: keyboard, mouse, storage	Internal devices: GPU, SSD, NIC	External high-speed docks, displays, storage
Hot-plug	Yes	Yes (with support)	Yes

USB is host-polled and optimized for low-cost, low-to-moderate bandwidth peripherals with broad compatibility. PCIe offers a dedicated, switched point-to-point fabric with the lowest latency and highest sustained throughput, making it the backbone for internal high-performance devices. Thunderbolt effectively tunnels PCIe (and DisplayPort) traffic over a single external cable, giving USB-C-style convenience with PCIe-class performance, which is why it is used to bridge external peripherals into the internal high-speed fabric.

Scalable Communication Model

A heterogeneous architecture uses a layered hub-and-switch model so any of the three buses can attach without redesigning the core:

- Root Complex / Host Controller Layer: A central PCIe root complex connects the CPU/memory to switch fabrics for GPU, storage, and network devices.
- Bridge Layer: USB host controllers and Thunderbolt controllers attach as PCIe endpoints, translating USB/Thunderbolt transactions into PCIe transaction-layer packets.
- Switch/Hub Layer: PCIe switches and Thunderbolt daisy-chain hubs allow tree expansion; USB hubs allow further fan-out for low-speed peripherals.
- Unified Address Space: All devices are mapped into a common physical address space via memorymapped I/O (MMIO) windows, so the OS can address any peripheral uniformly regardless of the underlying bus.
- Scalability Mechanism: New peripherals attach at any hub/switch port; the system enumerates them dynamically (PCIe configuration space, USB descriptors) and allocates bandwidth using bus-specific arbitration (PCIe

credit-based flow control, USB scheduled/isochronous transfers, Thunderbolt bandwidth reservation).

This model scales because each bus contributes independent bandwidth domains that merge at the root complex, and the OS's unified driver/MMIO abstraction hides the physical bus differences from applications.

Q2. Construct an interrupt handling framework for a multi-core processor comparing vectored interrupts and shared interrupt lines, and analyze latency impact.

Framework Design

- Interrupt Sources: Each I/O device raises an interrupt request (IRQ) to a local or global interrupt controller (e.g., APIC/GIC).

- Interrupt Controller: A central controller (I/O-APIC) receives IRQs and routes them to a specific core's Local APIC based on affinity/load-balancing policy.
- Vector Assignment: Each interrupt source is assigned a unique vector number that indexes directly into the core's Interrupt Descriptor Table (IDT), pointing straight to its ISR.
- Core Dispatch: The target core's Local APIC signals the CPU, which pushes context, looks up the vector in the IDT, and jumps to the corresponding Interrupt Service Routine (ISR).
- Multi-core Load Balancing: The controller can distribute interrupts across cores (e.g., round-robin or lowest-priority delivery) to avoid overloading a single core.

Vectored Interrupts vs Shared Interrupt Lines

Aspect	Vectored Interrupts	Shared Interrupt Lines
Identification	Device directly supplies a unique vector/ID	One line shared by multiple devices; source unknown
Service Routine Lookup	Direct jump via IDT/vector table	Requires polling all devices on the line to find the source
Latency	Low and near-constant ($O(1)$ dispatch)	Higher and variable ($O(n)$ polling of n devices)
Scalability	Scales well; more vectors as devices grow (MSI/MSI-X)	Degrades as more devices share a line

Hardware Cost	Needs more vector table entries / MSI support	Simple wiring, fewer controller pins
---------------	---	--------------------------------------

Vectored interrupts (as used by PCIe MSI/MSI-X) let each device deliver its own vector, so the ISR is entered directly with deterministic latency dominated by context-switch overhead alone. Shared interrupt lines require the OS to iterate through every device's status register to identify the interrupting device before handling it, adding a linear search cost proportional to the number of devices on that line. As core count and device count grow, vectored/MSI-X delivery scales far better because each core can receive dedicated vectors, avoiding contention and the polling overhead that dominates shared-line latency under heavy interrupt load.

Q3. Design a high-speed NVMe storage subsystem using PCIe lanes, DMA transfers, and memory-mapped registers, and justify performance improvements.

Subsystem Design

- Physical Link: The NVMe SSD connects via multiple PCIe lanes (e.g., x4 Gen4/Gen5), giving several GB/s of raw duplex bandwidth.
- Controller Registers: The NVMe controller exposes its command/status/doorbell registers through memory-mapped I/O (MMIO), letting the CPU issue commands with simple memory writes instead of slow port I/O instructions.
- Submission/Completion Queues: The host driver maintains multiple parallel Submission Queues (SQ) and Completion Queues (CQ) in system memory (up to 64K queues, 64K commands each), allowing many cores to submit I/O concurrently without lock contention.

-
- Doorbell Mechanism: After placing a command in an SQ, the driver writes to an MMIO doorbell register to notify the controller, which then fetches the command via DMA.
- DMA Data Transfer: The controller uses DMA (via PRP lists or Scatter-Gather Lists) to move data directly between the SSD and host memory buffers without CPU involvement, and issues an interrupt (MSI-X, vectored per queue) on completion.

Performance Justification

- Parallelism: Multiple queues per core eliminate the single-queue bottleneck of legacy AHCI/SATA, enabling massive command parallelism matched to multi-core CPUs.
- Reduced CPU Overhead: MMIO doorbells and DMA transfers remove the CPU from the data path, freeing cycles compared to programmed I/O.
- Lower Latency: Vectored MSI-X interrupts let each completion queue notify its owning core directly, avoiding shared-line polling delay.
 - Higher Throughput: Wider/faster PCIe lanes scale bandwidth linearly with lane count and generation, so a Gen5 x4 link far exceeds SATA's fixed 6 Gbps ceiling.

Together, MMIO for low-latency control, multi-queue DMA for parallel bulk transfer, and vectored interrupts for completion notification let NVMe fully exploit PCIe bandwidth, yielding order-of-magnitude IOPS and throughput gains over legacy storage interfaces.

Q4. Construct an I/O scheduling mechanism comparing cycle stealing DMA and burst mode DMA for continuous multimedia streaming.

Scheduling Mechanism

- Buffer Management: Double or ring buffering is used so one buffer is filled by DMA while the other is consumed by the CPU/renderer, ensuring continuous playback without stalls.
- DMA Controller Arbitration: A DMA controller requests the system bus from the CPU/memory arbiter; the arbitration policy (cycle stealing vs burst) determines how bus cycles are shared.
- Threshold-based Triggering: Streaming transfers are triggered when a buffer's fill level crosses a low-water mark, keeping the pipeline continuously fed.
- Interrupt-driven Refill: On buffer-empty/full events, the DMA controller interrupts the CPU (or a driver thread) to queue the next transfer.

Cycle Stealing vs Burst Mode DMA

Aspect	Cycle Stealing DMA	Burst Mode DMA
Bus Access Pattern	Takes one bus cycle at a time, interleaved with CPU	Takes the bus for an entire block transfer at once
CPU Impact	Minimal per-cycle stalling; CPU keeps running between steals	CPU is blocked/stalled for the full burst duration
Transfer Speed	Slower overall (frequent arbitration overhead)	Faster overall (fewer arbitration overheads, contiguous transfer)
Best Fit	Latency-sensitive, interactive workloads sharing the bus	Large contiguous streaming transfers with a dedicated bus window

Multimedia Streaming Suitability	Smoother CPU responsiveness but may not sustain very high data rates	Higher sustained throughput but can cause momentary CPU/bus starvation
----------------------------------	--	--

For continuous multimedia streaming, burst mode DMA is generally preferred for the bulk data movement because it minimizes per-transfer arbitration overhead and sustains the high, steady data rate audio/video buffers need. However, pure burst mode can starve the CPU and other bus masters for the burst's duration, causing jitter elsewhere in the system. A practical scheduler therefore uses burst transfers sized to the buffer chunk (e.g., one video frame or audio block) combined with cycle-stealing between bursts, giving high throughput for the stream while still allowing the CPU periodic bus access — balancing throughput against system responsiveness.

Q5. Design a multi-level I/O communication architecture combining memory-mapped I/O, interrupt handling, and advanced bus interfaces, and evaluate throughput.

Architecture Levels

- Level 1 — Physical/Bus Level: PCIe (with USB/Thunderbolt bridged in) forms the physical transport, providing switched, point-to-point links with per-lane bandwidth scaling.

Level 2 — Register/Control Level: Every device exposes control, status, and doorbell registers via memory-mapped I/O, so the CPU issues commands using ordinary load/store instructions into the device's address window.

-
- Level 3 — Data Movement Level: Bulk data transfer is offloaded to DMA engines using descriptor rings (submission/completion queues), so large transfers bypass the CPU entirely after being programmed via MMIO.
- Level 4 — Notification Level: Completion and error events are signaled via vectored interrupts (MSI-X), delivered directly to the requesting core to minimize dispatch latency.
- Level 5 — Software/Driver Level: The OS driver stack coordinates queue depth, interrupt affinity, and buffer allocation, presenting a unified API to applications regardless of the underlying bus.

Throughput Evaluation

Total achievable throughput is bounded by the minimum of: (1) the physical bus bandwidth (lane count x per-lane rate), (2) DMA engine efficiency (descriptor processing rate and burst size), and (3) interrupt/CPU servicing rate (how quickly completions are drained). Because MMIO handles only small, latency-sensitive control operations while DMA handles bulk data, the control path does not bottleneck the data path — throughput scales primarily with PCIe lane bandwidth and queue depth. Vectored interrupts prevent the notification level from becoming a bottleneck, since completions are routed directly to available cores rather than serialized through a shared line. In well-tuned systems this layered design lets realized throughput approach the theoretical PCIe link bandwidth, with overhead dominated only by descriptor/queue management rather than register-level polling.